Top 5 interview questions commonly asked in AI/ML Engineer role and detailed answers to help you crack it!

What is the difference between LLMs and traditional ML models?

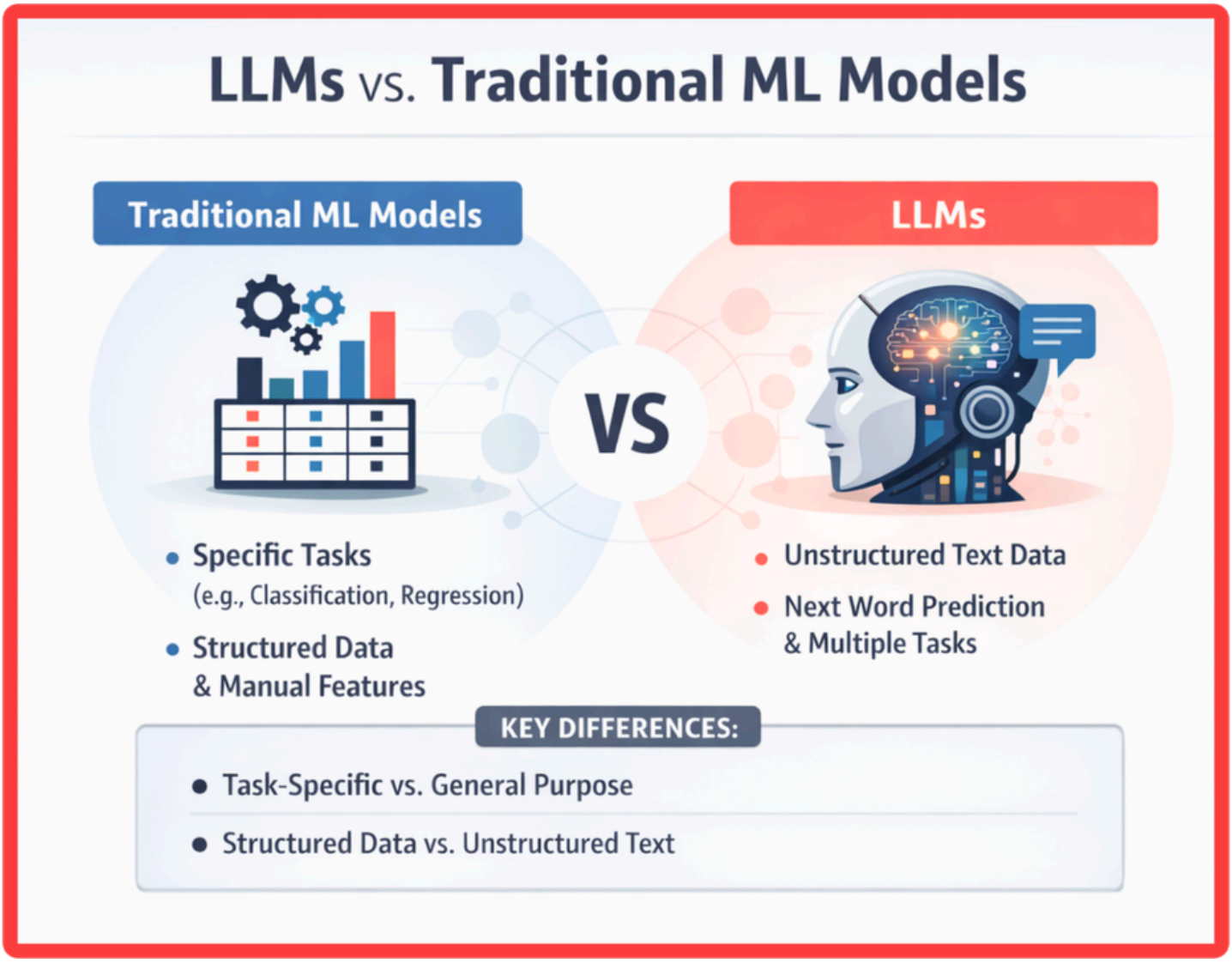
Answer:

Traditional ML models (like Logistic Regression, Random Forest, XGBoost) are trained for specific tasks such as classification or regression. They usually work on structured data (tables with rows and columns). Feature engineering is manually done before training.

Large Language Models (LLMs), on the other hand, are trained on massive unstructured text data. They are based on Transformer architecture and work by predicting the next token in a sequence. Instead of solving just one task, they can perform multiple tasks like summarization, translation, Q&A, and code generation.

Key Differences:

- Traditional ML → Task-specific, structured data
- LLM → General-purpose, unstructured text
- Traditional ML needs manual features
- LLM learns contextual representations automatically



What are Tokenization and Embeddings?

Answer:

Tokenization is the process of breaking text into smaller units called tokens. Tokens can be words, subwords, or even characters depending on the tokenizer (e.g., Byte Pair Encoding).

Example:

“Machine Learning is powerful”

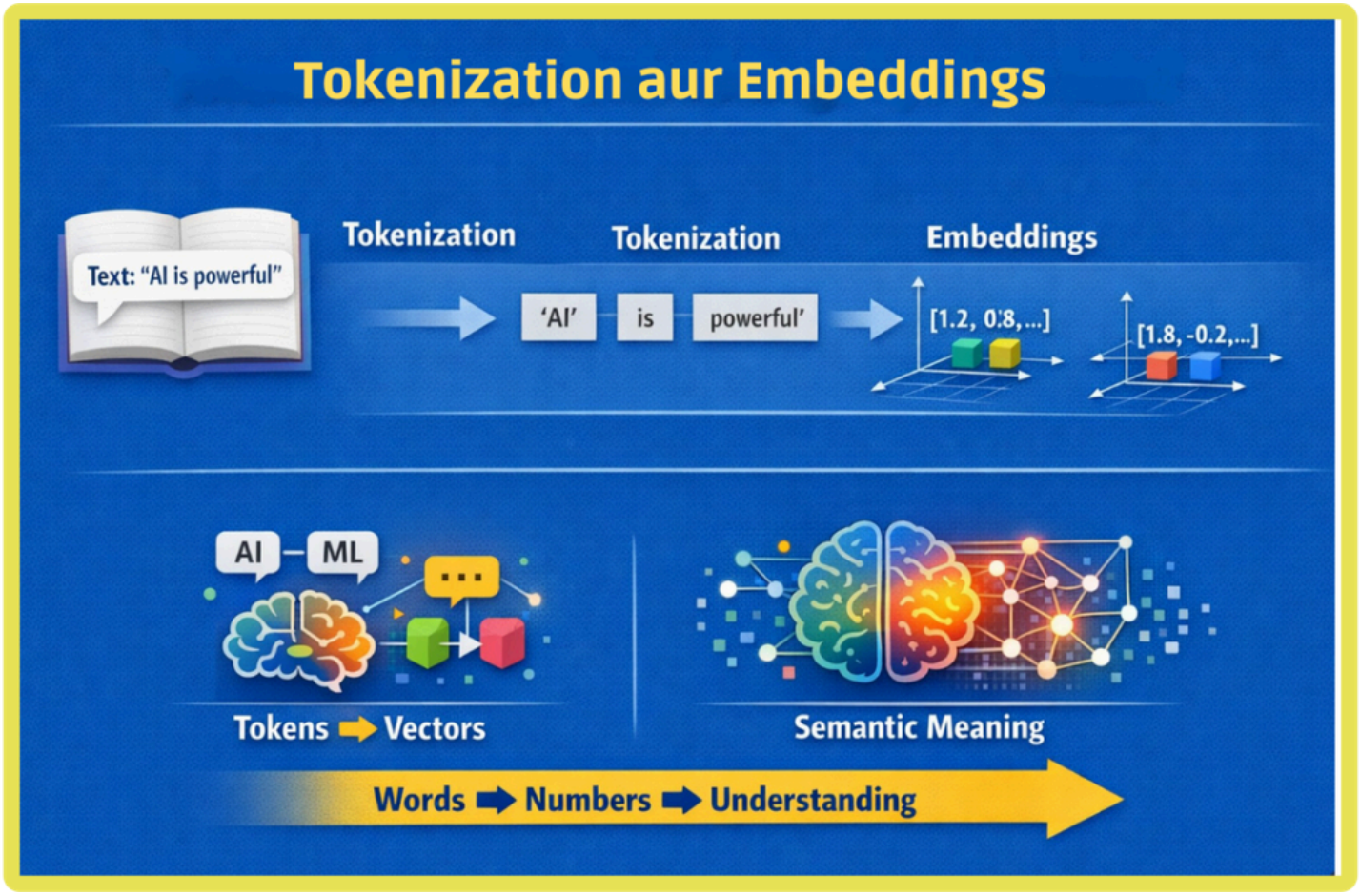
→ [“Machine”, “Learning”, “is”, “powerful”]

Embeddings are dense numerical vectors representing tokens in high-dimensional space. These vectors capture semantic meaning. Words with similar meanings have similar embeddings.

Flow:

Text → Tokenization → Token IDs → Embeddings → Model Processing

Embeddings help the model understand similarity, context, and relationships between words.



What is RAG (Retrieval-Augmented Generation)?

Answer:

RAG is a technique where we combine an LLM with external knowledge sources to generate more accurate and grounded responses.

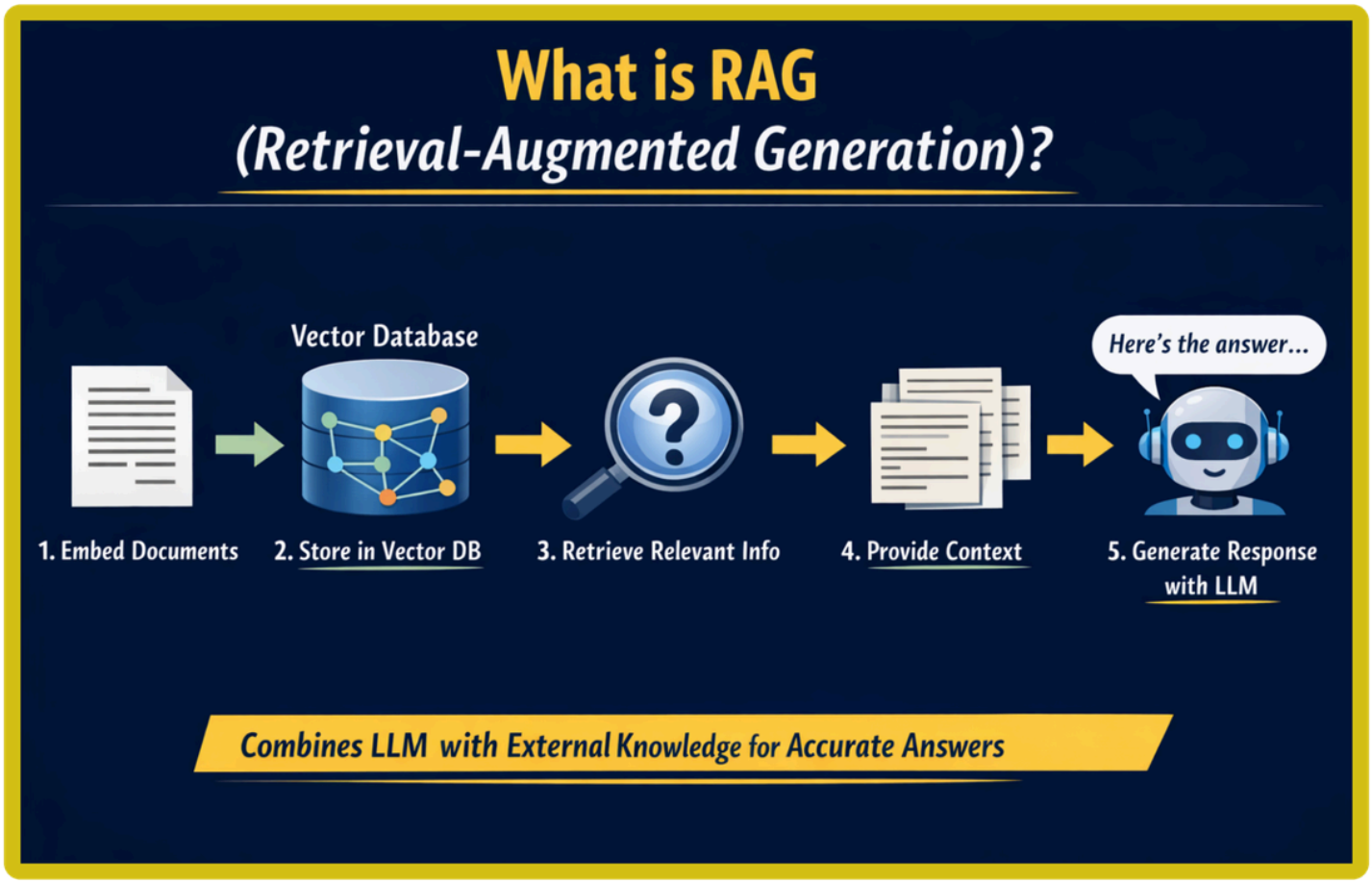
Instead of relying only on the model's training data, we:

1. Convert documents into embeddings
2. Store them in a vector database (like FAISS or Pinecone)
3. Convert user query into embedding
4. Retrieve top relevant documents
5. Pass retrieved context to LLM for final answer

Benefits:

- Reduces hallucination
- Keeps responses up-to-date
- Useful for enterprise chatbots and document Q&A systems

RAG is often preferred over fine-tuning for domain-specific knowledge.



Fine-tuning vs Prompt Engineering

Answer:

Prompt Engineering means carefully designing input instructions to guide the LLM's output without changing model weights.

Example:

Instead of asking "Explain AI,"

You say: "Explain AI in simple terms for a 10-year-old in 3 bullet points."

Fine-tuning means retraining the model (fully or partially) on domain-specific data to improve performance for specific tasks.

Comparison:

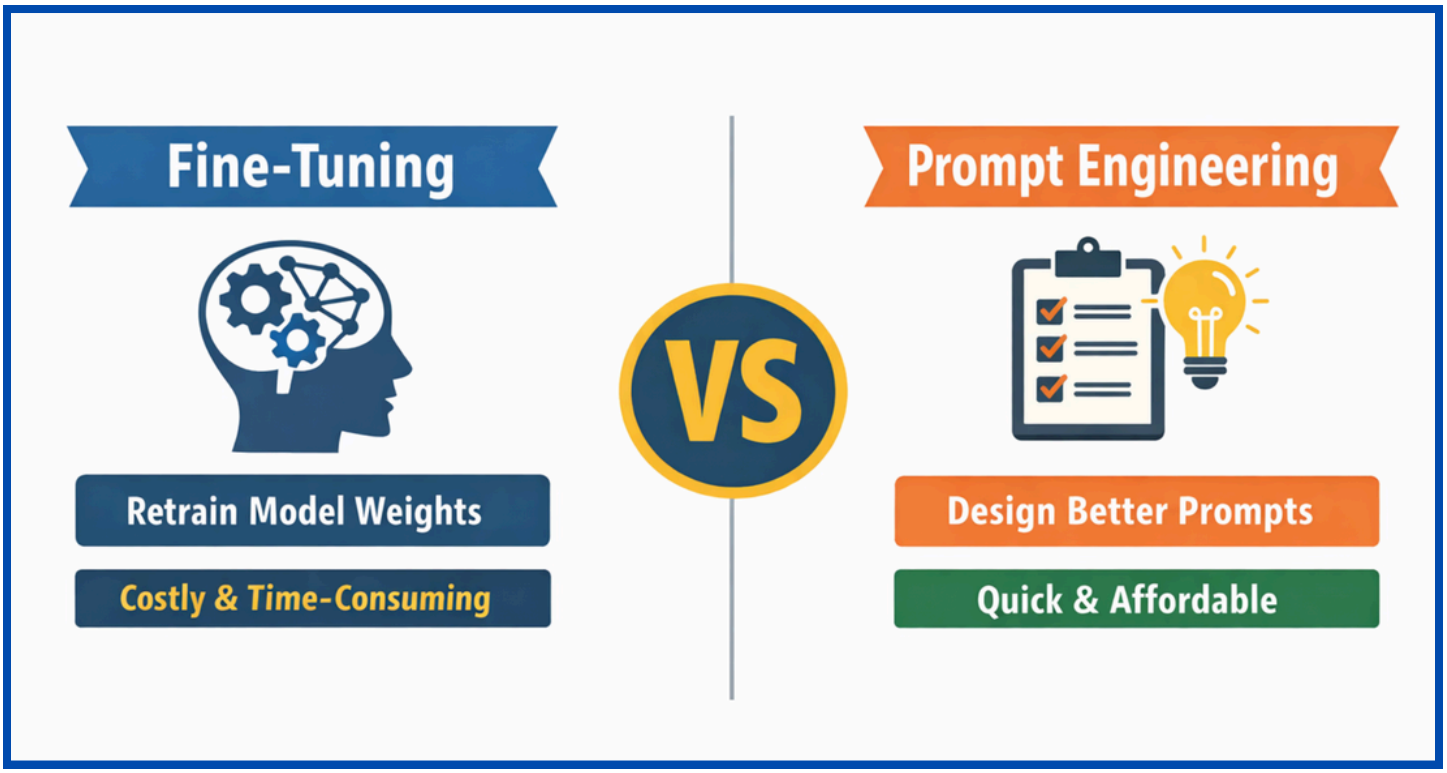
Prompt Engineering:

- Fast
- Low cost
- No retraining
- Good for most use cases

Fine-tuning:

- More control
- Better for niche or highly specific tasks
- Expensive and requires compute

In real-world projects, companies often start with prompt engineering or RAG before fine-tuning.



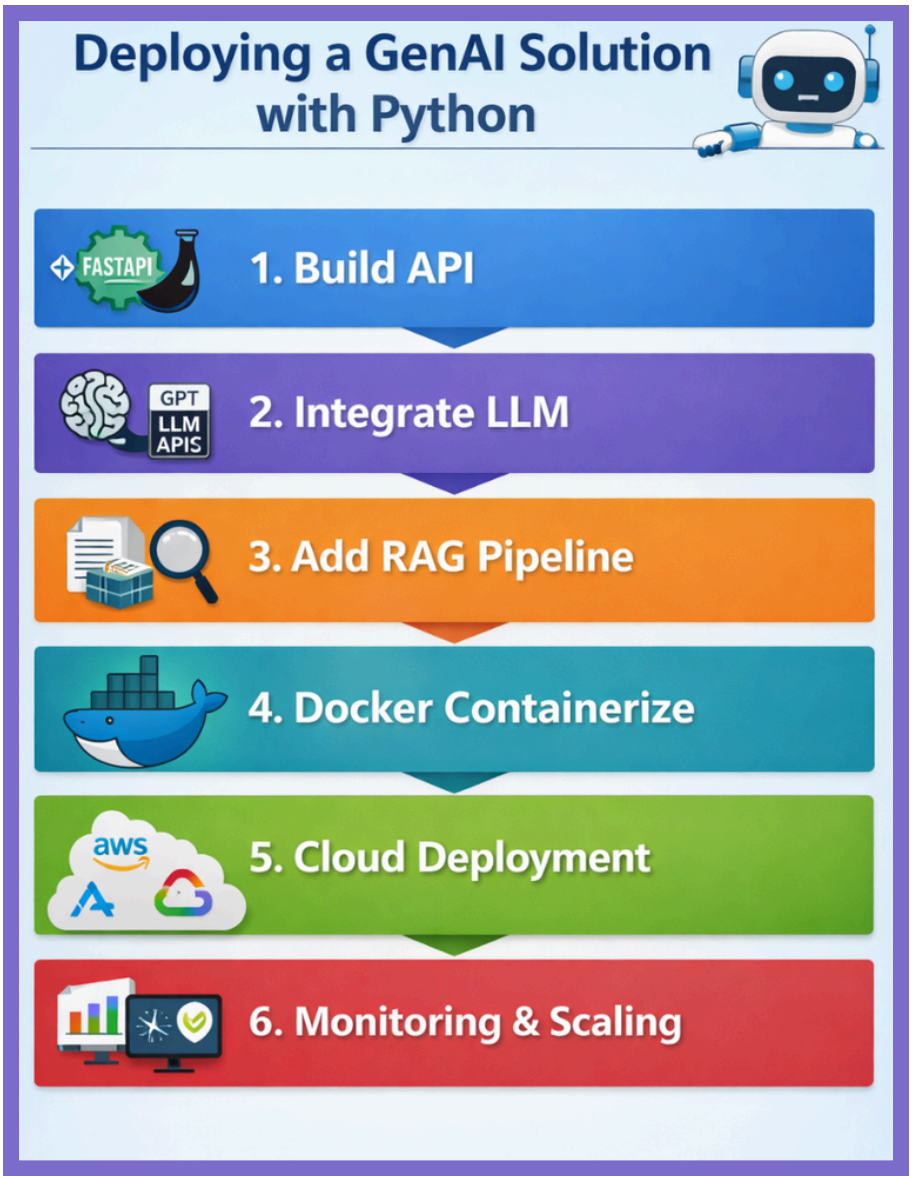
How would you deploy a GenAI solution using Python?

Answer:

A typical production-ready GenAI deployment includes:

1. Backend API:
2. Use FastAPI or Flask to create REST endpoints.
3. Model Integration:
4. Connect to OpenAI API or hosted LLM (or self-hosted model).
5. Business Logic:
6. Add prompt templates, RAG pipeline, or validation logic.
7. Containerization:
8. Use Docker to package the application.
9. Cloud Deployment:
10. Deploy on AWS, Azure, or GCP.
11. Monitoring:
12. Add logging, rate limiting, token usage tracking, and performance monitoring.

In interviews, always mention scalability, security (API keys), and monitoring — that shows production awareness.



If you're preparing for GenAI + Python interviews, follow/subscribe my channel(@learnwithbhawana) and level up your preparation.